

Examine alerts, logs, and rules with Suricata

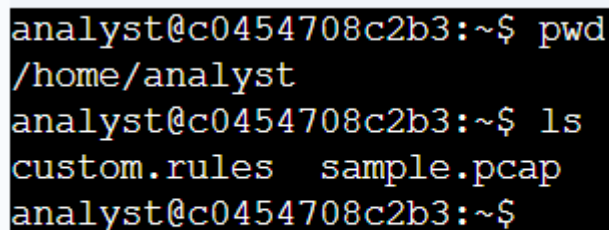
In this lab, I'll be required to configure Suricata and use it to trigger alerts.

First, I'll explore custom rules in Suricata.

Second, I'll run Suricata with a custom rule in order to trigger it, and examine the output logs in the fast.log file.

Finally, I'll examine the additional output that Suricata generates in the standard eve.json log file.

In order to get a general idea of my environment , I ran the command `pwd` , and `ls` to get the directory where I am at and the files present :

A terminal window with a black background and light blue text. The prompt is 'analyst@c0454708c2b3:~\$'. The first command is 'pwd', which outputs '/home/analyst'. The second command is 'ls', which outputs 'custom.rules' and 'sample.pcap'. The prompt returns to 'analyst@c0454708c2b3:~\$'.

```
analyst@c0454708c2b3:~$ pwd
/home/analyst
analyst@c0454708c2b3:~$ ls
custom.rules  sample.pcap
analyst@c0454708c2b3:~$
```

Here I am at the /home/analyst directory containing :

- custom.rules => suricata rules from the lab
- sample.pcap => a packet capture

Then , in order to read the customized rules , I ran the command `cat "filename"` to read the entire file or `less "filename"` to read it in another way page per page if too long to read easily.

```
analyst@c0454708c2b3:~$ cat custom.rules
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"GET on wire"; flow:established,to_server; content:"GET"; http_method; sid:12345; rev:3;)
analyst@c0454708c2b3:~$ less custom.rules
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"GET on wire"; flow:established,to_server; content:"GET"; http_method; sid:12345; rev:3;)
analyst@c0454708c2b3:~$
```

The command returns the rule as the output in the shell:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"GET on wire";
flow:established,to_server; content:"GET"; http_method; sid:12345;
rev:3;)
```

Custom Rule in Suricata

This rule consists of three components: an **action**, a **header**, and **rule options**.

Actions differ across network intrusion detection system (NIDS) rule languages, but some common actions are alert, drop, pass, and reject.

1. ALERT

- **Definition:** The system inspects the packet, matches it against a signature, generates a log entry (an "alert"), and **allows the packet** to reach its destination.
- **Deep Analysis:** This is "Passive Mode." It is used for monitoring and baseline creation.
- **The "Amazon" Perspective:** In a massive SOC, we use alert first to test new rules. If we jumped straight to blocking, a "False Positive" could accidentally take down the checkout page for millions of customers.
- **Consequence:** The attack reaches the target. You have a record of the crime, but you didn't stop it.

2. PASS

- **Definition:** The system ignores the packet and stops further rule processing for that specific traffic flow. The packet continues its journey.
- **Deep Analysis:** Think of this as a "Whitelist." It tells the NIDS: "I trust this specific traffic; don't waste CPU cycles checking it against 30,000 other rules."
- **Interaction:** A pass rule usually takes precedence over an alert or drop rule.
- **The Risk:** If an attacker spoofs an IP address that you have set to pass, they become invisible to your sensors.

3. DROP

- **Definition:** The system blocks the packet and **silently discards** it. No notification is sent back to the sender.
- **Deep Analysis:** This is the most "aggressive" defensive action. To the attacker, the connection simply "times out."
- **The Expert's "Pro Tip":** drop is superior during a **DDoS attack**. Because the system doesn't send a response, it saves bandwidth and processing power compared to reject. It also leaves the attacker guessing if the server is down or if there is a firewall in the way.

4. REJECT

- **Definition:** The system blocks the packet and **sends an error message** back to the sender (e.g., a TCP RST for TCP or an ICMP Destination Unreachable for UDP).
- **Deep Analysis:** It is an active "No."
- **Pros/Cons:** It allows legitimate applications to close the connection immediately (no waiting for a timeout). However, it confirms to a hacker that your security system is active and monitoring that specific port.

The Anatomy of a Rule Header

The header contains the "who, where, and how" of the traffic. It typically follows this fixed structure:

[Action] [Protocol] [Source IP] [Source Port] [Direction] [Destination IP] [Destination Port]

Let's break down each component:

- **Action:** (The 4 we just discussed: alert, log, pass, drop, reject). This tells the system what to do if the rest of the header and options match.
- **Protocol:** Usually tcp, udp, icmp, or ip. High-performance SOCs often use specific protocols to reduce "noise."
- **Source/Destination IP:** Can be a single IP (192.168.1.50), a CIDR block (192.168.1.0/24), or a variable like \$EXTERNAL_NET (everything outside your company) and \$HTTP_SERVERS.
- **Source/Destination Port:** Can be a specific number (80), a range (1:1024), or the keyword any.
- **Direction Indicator:** * -> : Traffic going from Source to Destination (most common).
 - <> : Bidirectional traffic (checks both ways, useful for tracking a full conversation).

The Anatomy of Rule Options

Rule options are always contained within **parentheses ()** and each parameter is separated by a **semicolon ;**. They consist of two parts: a **keyword** (like content) and an **argument** (like "GET").

1. The Metadata: msg, sid, and rev

- **msg (Message):** This is the "Title" of the alert. When you are a SOC analyst looking at your dashboard, this is the text you see. It must be clear (e.g., "Exploit Attempt - Log4j").
- **sid (Signature ID):** A unique ID.
 - *Expert Insight:* IDs below 1,000,000 are usually reserved for official system rules. Custom rules you write for your company should start at 1,000,001.
- **rev (Revision):** This is the version number. If you update the rule to make it more accurate, you increment the rev.

2. The Context: flow

- **flow:established,to_server:** This is one of the most powerful options. It tells the NIDS: "Only trigger this rule if the TCP 3-way handshake is already finished."
 - **Why?** An attacker might send a random packet with the word "GET" to a closed port. If you don't use flow:established, the NIDS will alert you for nothing. By using it, you only alert on **real** connections.

3. The Payload: content

- **content:"GET":** This searches the actual data of the packet for that specific string.
- **Case Sensitivity:** By default, "GET" is not the same as "get". You can add a modifier like nocase; to catch both.
- **Location:** You can tell the NIDS *where* to look (e.g., http_method). This makes the search much faster because the NIDS doesn't look at the whole packet, only the "Method" section of the HTTP header.

Run Suricata with a custom rule

```
analyst@c0454708c2b3:~$ ls -l /var/log/suricata
total 0
analyst@c0454708c2b3:~$
```

The Standard System Path (/var/log/)

In Linux, applications don't just put their logs wherever they want. There is a standard called the **Filesystem Hierarchy Standard (FHS)**.

- **/var/log/** is the "Central Records Office" of Linux.
- Suricata (the engine) runs in the background and sends its "reports" (logs) to **/var/log/suricata/**.

ls -l /var/log/suricata: I used an **Absolute Path** to look inside a specific "filing cabinet" across the room. The -l (long) flag is used here to see **permissions**—which is crucial because logs are often protected so that only "root" or the "suricata" user can read them.

For now there are no files in the directory .

Now , I run suricata with the pcap file and the custom rule file using :

```
sudo suricata -r sample.pcap -S custom.rules -k none
```

- The -r sample.pcap option specifies an input file to mimic network traffic. In this case, the sample.pcap file.
- The -S custom.rules option instructs Suricata to use the rules defined in the custom.rules file.
- The -k none option instructs Suricata to disable all checksum checks.

After cheking the suricata folder again with `ls -l /var/log/suricata` , 4 new files have been made :

```
analyst@c0454708c2b3:~$ sudo suricata -r sample.pcap -S custom.rules -k none
25/2/2026 -- 13:59:38 - <Notice> - This is Suricata version 6.0.1 RELEASE running in USER mode
25/2/2026 -- 13:59:39 - <Notice> - all 2 packet processing threads, 4 management threads initialized, engine started.
25/2/2026 -- 13:59:39 - <Notice> - Signal Received. Stopping engine.
25/2/2026 -- 13:59:39 - <Notice> - Pcap-file module read 1 files, 200 packets, 54238 bytes
analyst@c0454708c2b3:~$ ls -l /var/log/suricata
total 16
-rw-r--r-- 1 root root 1418 Feb 25 13:59 eve.json
-rw-r--r-- 1 root root 292 Feb 25 13:59 fast.log
-rw-r--r-- 1 root root 3239 Feb 25 13:59 stats.log
-rw-r--r-- 1 root root 1512 Feb 25 13:59 suricata.log
```

fast.log

By running the command `cat /var/log/suricata/fast.log`,

The output returns alert entries in the log:

Each line or entry in the fast.log file corresponds to an alert generated by Suricata when it processes a packet that meets the conditions of an alert generating rule.

Each alert line includes the message that identifies the rule that triggered the alert, as well as the source, destination, and direction of the traffic.

```
analyst@c0454708c2b3:~$ cat /var/log/suricata/fast.log
11/23/2022-12:38:34.624866  [**] [1:12345:3] GET on wire [**] [Classification: (
null)] [Priority: 3] {TCP} 172.21.224.2:49652 -> 142.250.1.139:80
11/23/2022-12:38:58.958203  [**] [1:12345:3] GET on wire [**] [Classification: (
null)] [Priority: 3] {TCP} 172.21.224.2:58494 -> 142.250.1.102:80
```

eve.json log

By running the command `cat /var/log/suricata/eve.json`, the output returns the raw content of the file. There is a lot of data returned that is not easy to understand in this format.

```
analyst@c0454708c2b3:~$ cat /var/log/suricata/eve.json
{"timestamp":"2022-11-23T12:38:34.624866+0000","flow_id":489190672988309,"pcap_c
nt":70,"event_type":"alert","src_ip":"172.21.224.2","src_port":49652,"dest_ip":
"142.250.1.139","dest_port":80,"proto":"TCP","tx_id":0,"alert":{"action":"allowed
","gid":1,"signature_id":12345,"rev":3,"signature":"GET on wire","category":"","
severity":3},"http":{"hostname":"opensource.google.com","url":"/","http_user_age
nt":"curl/7.74.0","http_content_type":"text/html","http_method":"GET","protocol
":"HTTP/1.1","status":301,"redirect":"https://opensource.google/","length":223},"
app_proto":"http","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver":3
57,"bytes_toclient":788,"start":"2022-11-23T12:38:34.620693+0000"}}
{"timestamp":"2022-11-23T12:38:58.958203+0000","flow_id":1804754189718772,"pcap
cnt":151,"event_type":"alert","src_ip":"172.21.224.2","src_port":58494,"dest_ip"
:"142.250.1.102","dest_port":80,"proto":"TCP","tx_id":0,"alert":{"action":"allow
ed","gid":1,"signature_id":12345,"rev":3,"signature":"GET on wire","category":"","
severity":3},"http":{"hostname":"opensource.google.com","url":"/","http_user_a
gent":"curl/7.74.0","http_content_type":"text/html","http_method":"GET","protoco
l":"HTTP/1.1","status":301,"redirect":"https://opensource.google/","length":223}
,"app_proto":"http","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver"
:357,"bytes_toclient":797,"start":"2022-11-23T12:38:58.955636+0000"}}}
```

By running `jq . /var/log/suricata/eve.json | less`, we can display the entries in an improved format:

```
analyst@c0454708c2b3:~$ jq . /var/log/suricata/eve.json | less
{
  "timestamp": "2022-11-23T12:38:34.624866+0000",
  "flow_id": 489190672988309,
  "pcap_cnt": 70,
  "event_type": "alert",
  "src_ip": "172.21.224.2",
  "src_port": 49652,
  "dest_ip": "142.250.1.139",
  "dest_port": 80,
  "proto": "TCP",
  "tx_id": 0,
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 12345,
    "rev": 3,
    "signature": "GET on wire",
    "category": "",
    "severity": 3
  },
  "http": {
    "hostname": "opensource.google.com",
```